

Edge-Cloud Vision System for Meta Quest 3S Using a Wi-Fi-Connected GPU Backend

Hoang Phuc Nguyen
Hung Luong Duc

CSE 422 Honors Option

Abstract

This project implements an end-to-end edge-cloud vision system for a Meta Quest 3S mixed-reality headset. The headset captures camera frames in Unity, compresses each frame as a JPEG image, and sends it over a Wi-Fi TCP connection to a local GPU backend. The Python backend decodes the image, runs YOLO segmentation on an NVIDIA GPU, and returns labels, bounding boxes, and optional mask polygons as newline-delimited JSON. Unity parses the response and renders labeled 2D bounding boxes over the camera preview. The system was demonstrated successfully in a recorded mixed-reality demo. A 100-frame localhost protocol sanity test, not a headset Wi-Fi latency result, achieved a median round-trip time of 27.98 ms with a persistent TCP connection. The live demo showed the main practical limitation: bounding boxes update with a small visible delay.

1 System Overview

The goal of the project was to build a working distributed vision pipeline that connects a resource-constrained XR client to a more powerful GPU backend. The final system uses the Meta Quest 3S as the client and a local laptop with an NVIDIA RTX 4070 Laptop GPU as the inference server.

The end-to-end pipeline is:

1. Unity on the Quest captures a passthrough camera frame.
2. The client downsizes the image to the target resolution and encodes it as JPEG.
3. The client sends a 4-byte little-endian payload length followed by the JPEG bytes over TCP.
4. The Python backend decodes the image and runs YOLO segmentation on the GPU.
5. The server sends one newline-terminated JSON response containing detections.
6. Unity parses the JSON and draws labeled bounding boxes over the preview image.

1.1 Network Topology

The final demo used an approved local GPU machine instead of the originally described campus GPU server. This changed the networking environment from remote campus/server routing to a local LAN Wi-Fi setup.

Stage	Component	Network/Data Role
Capture	Quest 3S Unity client	Produces camera frame for transmission.
Compression	Unity client	Encodes frame as JPEG to reduce payload size.
Request transport	TCP socket	Sends length-prefixed image payload reliably and in order.
Inference	Python GPU backend	Decodes request, runs model, builds detection response.
Response transport	TCP socket	Sends one JSON line per processed frame.
Visualization	Unity client	Draws labels and bounding boxes from server response.

Table 1: End-to-end architecture and data flow.

- **Client:** Meta Quest 3S headset [?].
- **Backend:** Lenovo laptop on the same Wi-Fi LAN.
- **Backend bind address:** 0.0.0.0:40081, so the service accepts remote LAN connections.
- **Observed LAN addressing during debugging:** Quest at 192.168.50.75/24, backend at 192.168.50.143/24.
- **Observed Wi-Fi link:** same SSID and 5 GHz channel, with Wi-Fi 6 reported by the headset diagnostics. Wi-Fi 6 is the 802.11ax generation of Wi-Fi [?].
- **NAT traversal:** not required, because the client and backend were on the same private subnet.

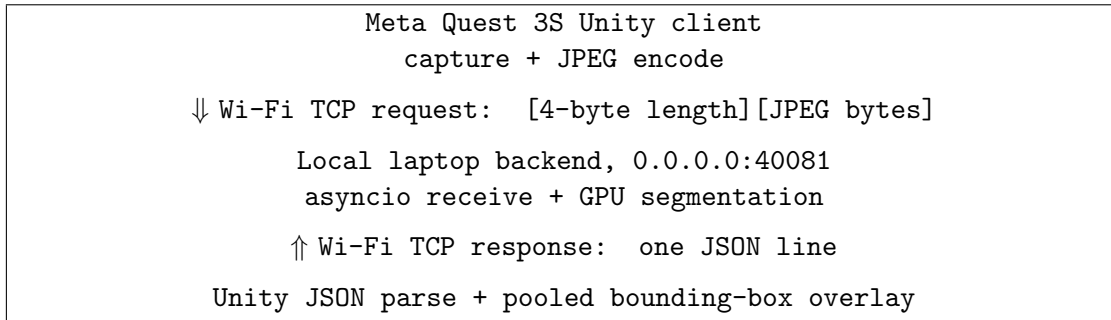


Figure 1: Network path and request/response flow for one processed frame.

2 Network Design

The network layer uses a custom TCP request/response protocol. TCP was chosen for this minimum viable prototype (MVP) because it gives [?]:

- reliable delivery of each complete image frame;
- ordered delivery, so responses stay associated with the frame order;
- simpler debugging than an unreliable transport;
- a natural request/response model: send one frame, receive one result.

The protocol uses explicit application-level framing:

- **Request header:** 4-byte little-endian integer containing the JPEG payload length.
- **Request body:** exactly that many JPEG bytes.
- **Response:** one UTF-8 JSON object terminated by a newline.

This framing is necessary because TCP is a byte stream, not a message protocol. The length prefix tells the server exactly how many bytes belong to the current image. The newline delimiter lets Unity find the end of the JSON response without needing a second response-length header.

The wire format is:

Client -> Server:

[4-byte little-endian length N] [N JPEG bytes]

Server -> Client:

```
{"frame_id":17,"status":"ok","latency_ms":31.2,"detections":[...]}\n
```

Important protocol and socket parameters are:

Parameter	Value / Purpose
Backend port	40081.
Server bind address	0.0.0.0, allowing LAN clients to connect.
Maximum frame size	10 MiB on the backend to reject malformed or oversized requests.
Client response cap	262,144 bytes to avoid unbounded JSON reads.
Client socket timeout	10,000 ms for connect/read/write operations.
TCP no-delay	Enabled on the Unity client; Python asyncio also enables <code>TCP_NODELAY</code> by default for TCP connections in modern Python [?].
Capture payload	640×480 JPEG, quality 60 in the Unity client.
Send interval	0.5 s by default, so the live client attempts about 2 processed frames per second.
Server sequence ID	Each response includes a server-generated <code>frame_id</code> and status field. The current request format does not include a client-generated frame ID or timestamp.

Table 2: Protocol and socket parameters used by the client/server system.

The 10,000 ms socket timeout is a safety timeout for detecting stalled connections. It is not the target latency for mixed reality. A production version should discard or fade responses that exceed a much smaller freshness threshold.

The client keeps one frame in flight at a time. This is an intentional flow-control decision:

- If Unity sent frames faster than the backend could process them, stale frames would queue up.
- Stale detections could then be drawn over the current live camera view.
- One frame in flight lowers throughput, but it keeps latency and visual correctness easier to reason about.

The Unity client also keeps a persistent TCP connection during a sending session:

- Reusing one connection avoids repeating TCP setup for every frame.
- The server can maintain naturally increasing frame identifiers across a run.
- If the socket fails, the client closes it and reconnects on the next send attempt.

One practical network issue occurred during integration:

- Windows marked the Wi-Fi network as a public network.
- The public firewall profile blocked inbound headset connections even though the backend was listening on port 40081.
- The server was reachable locally, but remote clients on the same subnet timed out.
- Setting the Wi-Fi network profile to Private and/or adding an inbound firewall rule for TCP port 40081 resolved the timeout.

The main tradeoff is reliability versus freshness:

- TCP gives reliable delivery and simple frame boundaries.
- Mixed-reality visuals are time-sensitive.
- If capture, encoding, network transfer, or inference takes too long, the bounding box can be correct for an older frame but appear delayed relative to the live view.

TCP head-of-line blocking is an important limitation. If a segment of a JPEG frame is lost, later bytes cannot be delivered to the application until the missing data is retransmitted. This is acceptable for the MVP because the client sends only one frame at a time, so there is no large queue of later frames waiting behind a lost packet. If the system allowed multiple frames in flight, TCP head-of-line blocking could make stale detections accumulate.

Transport	Advantage	Drawback
TCP	Reliable, ordered, simple to debug, easy request/response framing.	Head-of-line blocking; late frames are still delivered.
UDP	Can drop stale frames and avoid retransmission delay.	Requires custom reliability, ordering, and loss handling if needed.
WebSocket	Convenient message abstraction over TCP.	Still inherits TCP head-of-line blocking.
WebRTC/video	Designed for real-time media and congestion control [?].	More complex and harder to attach simple per-frame inference metadata.

Table 3: Transport tradeoffs for the MR vision pipeline.

3 Backend and Client Implementation

The backend implementation is responsible for receiving frames and running inference:

- Python asyncio TCP server.
- Reads length-prefixed image frames from each client.
- Decodes JPEG images with Pillow.
- Runs YOLO segmentation through Ultralytics [?].
- Loads the model once at server startup, not once per frame.
- Uses `--no-frame-log` during live runs to avoid disk I/O latency.

Each detection response includes:

- frame identifier and status;
- received byte count and decoded image size;
- model name and backend inference latency;
- detection class labels and confidence scores;
- bounding boxes in decoded image coordinates as `[x_min, y_min, x_max, y_max]`;
- optional segmentation mask polygons.

For the live headset overlay, Unity uses the labels and bounding boxes. Mask polygons are kept in the server response for debugging and offline annotated outputs, but live mask rendering is optional and not required for the demonstrated result.

The Unity client is responsible for capture, networking, and visualization:

- Runs on the Meta Quest 3S.

- Uses asynchronous GPU readback instead of the older blocking readback path [?].
- Encodes JPEG data on a background task.
- Uses a persistent `TcpClient`.
- Parses JSON responses with Newtonsoft JSON.
- Draws pooled UI bounding boxes and labels over the preview image.

3.1 Overlay Coordinate Mapping

The server returns bounding boxes in decoded image pixel coordinates:

- origin at the top-left of the decoded image;
- x increasing to the right;
- y increasing downward;
- coordinates measured against the server-reported image width and height.

Unity maps those boxes onto the preview UI as follows:

- Normalize each coordinate by the decoded image width and height.
- Compute the displayed image rectangle inside the `RawImage` using the image aspect ratio and the UI container aspect ratio.
- Apply letterbox or pillarbox offsets before placing the overlay.
- Convert from image top-left coordinates to Unity UI coordinates by flipping the vertical axis.
- Apply optional mirror-X, mirror-Y, and rotation calibration values for headset/camera orientation differences.
- Place the label at the top-left of the box, moving it inside the box if the label would go above the preview.

The demo calibration values were:

- mirror-X disabled;
- mirror-Y disabled;
- overlay rotation set to none;
- mask polygon drawing disabled for the live demo.

3.2 Test Coverage

The backend repository includes automated tests for the main protocol and server behavior:

- frame-length packing and unpacking;
- JSON serialization for detections, empty detections, and error responses;
- fake-detector tests that avoid requiring a GPU during unit tests;
- async server integration tests that send an encoded image and verify the JSON response;
- evaluation-script helper tests for CSV/JSON output and TCP response parsing.

4 Evaluation

4.1 Hardware and Demo

The evaluation used the following hardware:

- **Headset:** Meta Quest 3S.
- **Backend machine:** Lenovo laptop.
- **CPU:** AMD Ryzen 7 8845HS.
- **Memory:** 32 GB RAM.
- **GPU:** NVIDIA GeForce RTX 4070 Laptop GPU.
- **Inference device:** CUDA was available during evaluation [?].

The instructor allowed the use of the local computer for the final implementation.

The recorded demo confirmed the end-to-end system:

- The submitted demo video is about 94.1 seconds long at 2560×1368 resolution.
- Demo video submitted with the report: `Casting _ Meta Horizon - Google Chrome 2026-05-05 20-18-44.mp4`.
- Unity sent frames from the Quest to the backend.
- The backend returned detection results.
- Unity rendered labeled bounding boxes in mixed reality.
- No crashes, socket failures, or visible parsing/rendering failures were observed in the recorded demo.
- The visible limitation was that bounding boxes took a small amount of time to update.

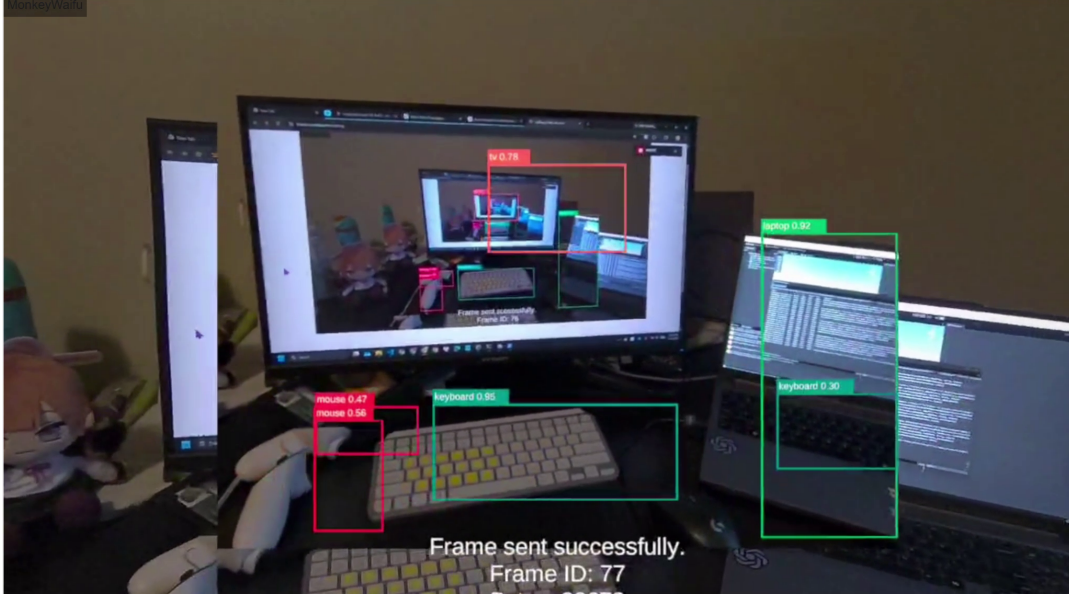


Figure 2: Frame extracted from the recorded demo showing the live Unity overlay with labels and bounding boxes.

One captured frame from the demo showed a readback time of about 41.8 ms. This should be interpreted as GPU readback wall time, not necessarily a full main-thread stall, because the implementation yields while waiting for asynchronous GPU readback.

The backend and evaluation test suite passed:

Ran 18 tests OK

4.2 Measurement Scope

The evaluation separates three different types of evidence:

Measurement	What it Shows	Limitation
Recorded Quest demo	Confirms real Wi-Fi end-to-end functionality and visible overlay rendering.	Does not provide per-frame RTT, jitter, or retransmission logs.
Saved-frame inference	Measures GPU/backend inference and response size without network variability.	Does not include headset capture or Wi-Fi transfer.
Local TCP sanity test	Measures protocol overhead, request/response stability, and persistent-vs-reconnect behavior.	Runs on 127.0.0.1, so it is not a Wi-Fi performance test.

Table 4: Scope of the collected measurements.

The main missing network measurement is a live per-frame Wi-Fi log from the Quest. The demo proved the Wi-Fi path worked, but it did not record enough samples to compute live wireless RTT, jitter, packet retransmissions, or reconnect rate. For that reason, the report does not claim

that the localhost TCP numbers represent headset Wi-Fi performance. This remains the largest evaluation weakness.

A small future headset-side log would close the biggest evaluation gap:

- server sequence ID from the response;
- Unity timestamp immediately before socket write;
- Unity timestamp immediately after the JSON line is received;
- server-reported processing latency from the JSON response;
- client-observed RTT and reconnect/failure status for each frame.

4.3 Saved-Frame Inference

Saved camera frames from the Quest were used to measure backend inference latency and response size without live headset or Wi-Fi variability. The first frame includes model/CUDA warm-up, so median latency and warm-up-excluded mean latency are more representative than the raw mean.

The saved JPEG request sizes were:

- mean JPEG size: 43,719 bytes;
- median JPEG size: 38,283 bytes;
- minimum JPEG size: 22,794 bytes;
- maximum JPEG size: 59,910 bytes.

Model	Frames	Median (ms)	Mean* (ms)	Avg. Resp. (bytes)	Avg. Det.
yolo26s-seg.pt	19	30.75	32.45	5,690.84	2.105
yolo26n-seg.pt	19	36.80	37.09	4,676.00	2.368

Table 5: Saved-frame backend inference results. The mean latency column excludes the first warm-up frame.

The filenames `yolo26s-seg.pt` and `yolo26n-seg.pt` are the Ultralytics YOLO26 segmentation checkpoints used in this project, not custom names or report shorthand [?]. In this dataset, `yolo26s-seg.pt` had the better steady-state latency, while `yolo26n-seg.pt` produced slightly smaller responses on average. Because this was only a 19-frame saved-frame run with GPU warm-up and scheduling effects, this result should not be interpreted as a general model-speed ranking. Response size varied depending on the number of detections and the size of segmentation mask polygons.

4.4 Bandwidth Demand Estimate

The Unity client uses a default send interval of 0.5 s, or about 2 processed frames per second when the backend keeps up. Using the measured persistent TCP request/response sizes, the approximate application-level bandwidth demand is:

Uplink Mbps = frames/s $\times$ request bytes $\times 8/1,000,000$. Downlink Mbps = frames/s $\times$ response bytes $\times 8/1,000,000$. The response sizes used here are from the current segmentation JSON response, which can include mask polygons; a boxes-only live response would reduce downlink bandwidth.

Processed Frame Rate (frames/s)	Uplink (Mbps)	Downlink (Mbps)	Total (Mbps)
2	0.695	0.088	0.783
5	1.737	0.220	1.956
10	3.473	0.440	3.913

Table 6: Estimated application-level bandwidth demand from measured request and response sizes.

These numbers are small compared with typical Wi-Fi 6 link capacity. This suggests that bandwidth was unlikely to be the primary bottleneck at the current frame rate. The more important network concerns are delay, jitter, retransmission behavior, and whether late results become visually stale.

4.5 Local TCP Protocol Sanity Test

A local TCP test sent saved frames to the running backend using the same protocol as Unity:

- 4-byte little-endian frame length;
- JPEG payload bytes;
- newline-terminated JSON response.

This test was run on 127.0.0.1. It measures backend protocol overhead and server stability, not real headset Wi-Fi latency.

Mode	Frames	Success	Mean RTT (ms)	Median RTT (ms)	P95 RTT (ms)	Max RTT (ms)
Persistent TCP	100	100/100	30.82	27.98	37.56	85.85
Reconnect each frame	100	100/100	33.27	29.65	61.42	76.29

Table 7: Local TCP protocol sanity test RTT statistics using saved frames.

Mode	Backend Median (ms)	Avg. Request (bytes)	Avg. Response (bytes)
Persistent TCP	22.12	43,417.29	5,493.80
Reconnect each frame	22.99	43,417.29	5,492.87

Table 8: Backend and payload statistics from the same local TCP protocol sanity test.

Both modes completed 100 out of 100 requests successfully. The persistent connection had lower mean and median round-trip time in this local test. This difference may matter more over Wi-Fi and should be tested with headset-side logging.

Because the local test does not measure Wi-Fi, the correct interpretation is:

- the application protocol handled 100 sequential requests without parse errors or socket failures;
- persistent TCP had lower typical RTT than reconnecting per frame in this test;
- real Wi-Fi latency, jitter, retransmissions, and reconnect behavior still require a headset-side logging run.

4.6 Reliability Observations

The reliability evidence collected so far is application-level:

- The recorded Quest demo completed successfully with no crashes, socket failures, or visible parsing/rendering failures observed.
- The local persistent TCP sanity test completed 100/100 requests successfully.
- The local reconnect-per-frame comparison also completed 100/100 requests successfully.
- No live packet-capture data was collected, so Wi-Fi packet loss and TCP retransmission rates were not measured directly.

5 Discussion and Limitations

The system met the core end-to-end requirement, but several latency sources remain:

- Camera readback on the headset contributes noticeable delay before the image is sent.
- JPEG encoding, TCP transfer, GPU inference, and JSON parsing all add pipeline latency.
- The one-frame-in-flight design intentionally limits throughput to avoid stale queues.
- The default 0.5 s send interval caps the overlay update rate at about 2 processed frames per second, so the interval itself can add up to about 500 ms between processed frames independent of readback, inference, or network latency.

From a networking perspective, TCP helped the MVP because:

- request and response boundaries are deterministic;
- the protocol is easy to debug;
- one persistent connection supports repeated frames without changing the wire format.

The drawback is that TCP does not solve visual staleness. If the pipeline runs slower than the scene changes, the delivered result can still arrive too late for a perfectly aligned mixed-reality overlay.

Response size is another limitation:

- The server can return segmentation mask polygons as well as bounding boxes.
- In crowded scenes, mask polygons can make the JSON response much larger.

- A future live mode could return boxes and labels by default while keeping masks available for debugging or offline analysis.

The live demo did not collect packet-level wireless statistics. Since TCP retransmits lost packets below the application layer, the Unity application cannot directly observe Wi-Fi packet loss unless extra tracing is added. A better future network evaluation would log:

- application-level frame failures and socket timeouts;
- reconnect attempts and successful reconnects;
- per-frame client-observed RTT from Unity;
- server-reported processing latency for the same frame;
- estimated non-server-processing overhead as client RTT minus server-reported processing time;
- per-stage timings for GPU readback, JPEG encode, socket send, server receive/decode, inference, JSON serialization, socket receive, JSON parse, and UI render;
- TCP retransmissions using Wireshark, Windows network tracing, or router statistics.

The non-server-processing overhead would still include several costs unless more timestamps are added. It can include Wi-Fi transfer, TCP buffering, Unity read/write timing, JSON parsing, and UI update work. A true network-only delay measurement would require tighter timing boundaries or packet capture.

Future work should focus on latency-aware networking and measurement:

- live boxes-only response mode;
- adaptive frame rate control based on observed round-trip time;
- more detailed headset-side logging across many frames;
- experiments with lower-latency transports such as UDP, WebRTC, or video streaming if the application requires higher update rates.

6 Conclusion

This project implemented a working edge-cloud mixed-reality vision pipeline. The Meta Quest 3S client captures frames, sends them to a GPU backend over a custom TCP protocol, receives detection results, and visualizes labels and bounding boxes in the headset. The networking contribution is the reliable length-prefixed TCP transport, newline-delimited JSON response protocol, persistent connection design, and one-frame-in-flight flow control. The recorded demo confirmed end-to-end functionality, while evaluation showed stable 100-frame TCP tests and backend inference latencies around tens of milliseconds after warm-up. The main remaining issue is latency: bounding boxes update slightly after the live scene changes, which is expected in a Wi-Fi edge-computing vision pipeline and is the primary target for future optimization.